Lecture 39: Deep Learning (Part-4) — Practice Problems Suite

This problem suite covers the foundations of Computer Vision, Convolutional Neural Networks, mathematical convolutions, spatial geometry (padding and stride), pooling layers, and complete CNN architectures.

Problem 1: Inductive Bias and Deep Learning Taxonomy

Compare the fundamental architectural inductive biases of Feedforward Neural Networks (FNNs), Convolutional Neural Networks (CNNs), and Recurrent Neural Networks (RNNs). Why is each architecture uniquely suited to tabular data, spatial visual grids, and temporal sequences, respectively?

Problem 2: Digital Image Matrix Representation

A high-resolution color camera captures an image of resolution 1920×1080.

1. Express the complete dimensionality of this tensor in both channels-last (H, W, C) and channels-first (C, H, W) formats.
2. How many total intensity scalar values does this single image contain?
3. What are the RGB intensity values for pure yellow, pure cyan, and 50% medium gray?

Problem 3: Mathematical Rationale for Pixel Normalization

Why must raw pixel intensities $\in [0, 255]$ be normalized to floating-point values $\in [0.0, 1.0]$ via $x_{\text{norm}} = x/255.0$ before being fed into a neural network? What happens to the loss landscape and gradient descent updates if unnormalized 8-bit integers are passed directly?

Problem 4: Computer Vision Task Spectrum

Contrast the 4 primary computer vision tasks:

1. Image Classification
2. Classification with Localization
3. Object Detection
4. Semantic Segmentation Specify the exact mathematical output (labels, coordinates, masks) produced by each task.

Problem 5: Real-World Multi-Task Vision Architecture

Autonomous vehicles (e.g. Tesla or Waymo) must navigate dynamic roadways in real time. Describe how a self-driving perception system integrates multiple computer vision tasks simultaneously across camera streams to detect lane markings, pedestrians, road signs, and potholes.

Problem 6: The Mathematical Catastrophe of Parameter Explosion

Suppose we attempt to train a standard Feedforward Neural Network (ANN) on full-HD RGB images of shape $1920 \times 1080 \times 3$. If the first hidden layer has 256 neurons:

1. Calculate the exact number of weights and biases in this single layer.
2. Explain why this model is impossible to train effectively, referencing memory limits and the bias-variance tradeoff.

Problem 7: Spatial Locality and Translation Invariance

Explain the concepts of **Spatial Locality** and **Translation Invariance**. How does flattening a 2D matrix into a 1D vector permanently destroy the spatial adjacency between (i, j) and $(i+1, j)$?

Problem 8: The Two-Stage CNN Blueprint

Diagram and explain the two foundational stages of a Convolutional Neural Network:

- Stage 1: Feature Extraction Engine
- Stage 2: Dense Classification Head What type of data representation flows between Stage 1 and Stage 2?

Problem 9: Hierarchical Visual Feature Learning

How does a deep CNN mirror biological visual processing? Trace the semantic evolution of extracted features from early convolutional layers (low-level), through middle layers (mid-level), to deep layers (high-level).

Problem 10: Manual 2D Convolution Calculation

Given the 4×4 input matrix X and 2×2 kernel K below (with bias $b = 1$):

$$X = \begin{bmatrix} 2 & 0 & 1 & 3 \\ 1 & 2 & 0 & 2 \\ 0 & 1 & 3 & 1 \\ 2 & 1 & 0 & 2 \end{bmatrix}, \quad K = \begin{bmatrix} 1 & -1 \\ 2 & 0 \end{bmatrix}, \quad b = 1$$

Assuming stride $S = 1$ and valid padding $P = 0$:

1. Determine the spatial dimensions of the output feature map.
2. Calculate the exact numerical output values for the top-left cell $(0, 0)$ and the top-right cell $(0, 2)$.

Problem 11: Learnable Filters vs Classical Image Processing Filters

In classical computer vision, edge detection relied on handcrafted filters like the Sobel filter:

$$K_{\text{Sobel}} = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Contrast classical handcrafted filters with convolutional kernels in a deep CNN. How are CNN kernel values determined?

Problem 12: The Fundamental Spatial Dimension Formula

State the universal formula for computing output feature map dimensions given input width W, kernel size K, padding P, and stride S.

1. Compute the output size for $W = 32, K = 5, P = 2, S = 1$.
2. Compute the output size for $W = 224, K = 7, P = 3, S = 2$.
3. Prove that for any odd kernel size K, setting $P = (K - 1)/2$ with $S = 1$ preserves the input spatial dimensions exactly ("Same Padding").

Problem 13: Pooling Layer Theoretical Properties

Explain why the Pooling Layer is essential in deep CNNs.

1. Perform 2×2 Max Pooling with stride $S = 2$ on the following 4×4 feature map:

$$\begin{bmatrix} 4 & 9 & 1 & 3 \\ 8 & 2 & 7 & 5 \\ 3 & 1 & 14 & 10 \\ 6 & 0 & 2 & 12 \end{bmatrix}$$

2. Prove that a pooling layer has exactly zero learnable parameters ($\text{params} = 0$).
3. How does Max Pooling provide translation invariance to small spatial perturbations?

Problem 14: Flattening, Dense Reasoning, and Dropout

1. If the final pooling layer outputs a feature tensor of shape $(128, 7, 7)$, what is the length of the vector produced by `Flatten()`?
2. Why is the dense layer described as performing "global non-linear reasoning"?
3. How does Dropout ($p = 0.5$) prevent overfitting in the dense classification head?

Problem 15: End-to-End Architectural Parameter Audit

Consider a complete CNN with the following sequential specification:

- Input: $(3, 32, 32)$ (CIFAR-10 RGB image)
- Layer 1: `Conv2d(3, 32, kernel_size=3, padding=1, stride=1)` + `ReLU`
- Layer 2: `MaxPool2d(kernel_size=2, stride=2)`
- Layer 3: `Conv2d(32, 64, kernel_size=3, padding=1, stride=1)` + `ReLU`
- Layer 4: `MaxPool2d(kernel_size=2, stride=2)`
- Layer 5: `Flatten()`
- Layer 6: `Linear(in_features=?, out_features=128)` + `ReLU`
- Layer 7: `Linear(128, 10)` Derive:

1. The output spatial shape after each layer.
2. The exact number of trainable parameters in each layer.
3. The total trainable parameters of the entire network.